What this project does

Two families of neural PDE solver on the 1D elastic wave equation

SciML Project — wave equation

August 5, 2026

Summary. One physics problem — the 1D elastic wave equation in heterogeneous media — is attacked with two unrelated families of neural solver, both measured against the same finite-difference reference. *Arm A* trains physics-informed coordinate networks (seven architectures, two compute backends, a 528-run sweep) that learn $(x, t) \mapsto u(x, t)$ for a single material from the PDE residual alone. *Arm B* trains neural operators (FNO, DeepONet, PFNO) that learn the map from an entire material profile to the complete wavefield, supervised on FD solutions, and generalize to media never seen in training. Arm B is where the measured results are: after diagnosing an under-specified training set, validation error falls from 81.9% to **1.96**% (FNO), 94.8% to **4.03**% (PFNO) and 97.3% to **24.57**% (DeepONet) on synthetic media, and the same architectural ranking survives on three published geological velocity models. Along the way: an unconverged reference target was found to have flattered one architecture by ten points, the worst model was found to score the best boundary residual, and adding physics losses was found to help a model that already fits while pushing an underfitting one further toward collapse. This document is the map — what exists, what was found, and where it lives.

Contents

1 The two arms **2**

2 The shared foundation **2**

3 Arm A — PINN / KAN coordinate networks **3**
- 3.1 Architectures . . . 3
- 3.2 Training machinery . . . 3
- 3.3 The sweep . . . 3
- 3.4 Status . . . 4

4 Arm B — neural operators **4**
- 4.1 The unforced arm, and the failure that started it . . . 4
- 4.2 The target was not converged . . . 5
- 4.3 The forced arm: a source term and physics residuals . . . 5
- 4.4 Making the physics an actual training loss . . . 5
- 4.5 Real velocity models . . . 6
- 4.6 Visualization . . . 7

5 What the whole thing found **8**

6 Repository map **8**

7 Status and honest limits **9**

1 The two arms

The governing equation is

$$\rho(x)\, u_{tt} \;=\; \frac{\partial}{\partial x}[E(x)\, u_x]\,, \qquad x \in [-1,1], \quad t \in [0,1]. \tag{1}$$

The two arms share nothing but this equation and the reference solver.

	Arm A — coordinate networks	**Arm B — neural operators**
learns	$(x,t) \mapsto u(x,t)$ for *one* material	$[E(x), \rho(x), \dots] \mapsto u(x,t)$ for *any* material
supervision	PDE residual (no solution data)	FD solutions (supervised)
cost model	retrain per problem	train once, infer in one forward pass
question	which architecture / optimizer / weighting trains a PINN best?	which operator architecture generalizes to unseen media?
models	VanillaPINN, FourierFeaturePINN, PirateNet, WavKAN, KAN, ChebyshevKAN, FourierWavKAN	FNO, DeepONet, PFNO
status	infrastructure complete; 528-run sweep runs on Sherlock	**run, measured, documented**

Arm B is where the results are. Its full write-up — physics, architectures, diagnosis, per-arm results, visualization rationale, limitations — is `docs/operator_simulations.pdf`. What follows is the map.

2 The shared foundation

The equation is solved in conservative form. Expanding gives $\rho u_{tt} = E u_{xx} + E_x u_x$, so the naive $u_{tt} = c(x)^2 u_{xx}$ drops the $E_x u_x$ term — exactly the term that produces correct reflection and transmission amplitudes at a material interface. Everything in the repository uses the conservative form.

The FD reference (`wave/fd_solver.py`) is an explicit second-order leapfrog scheme in flux form. Two details are load-bearing, and both were arrived at by fixing a failure:

- **Harmonic-mean interface stiffness** $E_{i+1/2} = 2E_iE_{i+1}/(E_i + E_{i+1})$, which preserves transmission and reflection across a jump in E.
- **Mur discretization** of the absorbing boundaries, $\beta = (c\,\Delta t - \Delta x)/(c\,\Delta t + \Delta x)$. The apparently natural centred-time / one-sided-space ABC is *unconditionally unstable*.

The reference has to be grid-converged. This was not true initially, and it turned out to matter (§4.2). Measured against a `refine=32` solve:

refine	**FD grid**	**relative L_2 vs converged**	**cost / 512 samples**
1	64	13.35%	1.8 s
4	253	1.14%	8.9 s
8	**505**	**0.32%**	**22 s**
16	1009	0.08%	41 s

The problem definition — initial condition, material sampler, refined solve, input-channel layout — lives in `wave/wave_problem.py`, deliberately torch-free so that dataset generation runs on machines with no GPU stack.

3 Arm A — PINN / KAN coordinate networks

Seven architectures compared on three canonical materials (Homogeneous, TwoLayer, MultiLayer), trained purely on PDE residuals.

3.1 Architectures

Model	Key idea
`VanillaPINN`	MLP, Tanh + Xavier initialization
`FourierFeaturePINN`	trainable random Fourier embedding → MLP
`PirateNet`	fixed Fourier embedding → RWF layers → residual blocks (α init 0)
`WavKAN`	wavelet KAN (Mexican Hat / Morlet / DoG / Meyer / Shannon)
`KAN`	spline KAN via `pykan` / `jaxkan`
`ChebyshevKAN`	cPIKAN — tanh-squashed inputs, Chebyshev recurrence per edge
`FourierWavKAN`	Fourier embedding → WavKAN trunk (spectral-bias fix for sharp interfaces)

Both backends implement all seven. PyTorch is the original; JAX/Equinox is a parallel reimplementation (`src/*_jax.py`) and is now the default. The JAX path computes the PDE residual with `jax.grad(jax.grad(·))` + `vmap` rather than reverse-over-reverse autograd, and uses `segment_sum` for causal chunking instead of a Python loop.

3.2 Training machinery

All of the following is implemented and switchable per run:

- **Two-phase schedule** — Adam (15 000 steps) then L-BFGS (2 000), or L-BFGS-only (5 000). L-BFGS runs with causal weighting *off*: weights changing inside the closure would corrupt the line-search objective.
- **Causal weighting** (Wang et al. 2022) — 16 time slabs, slab m weighted by $\exp(-\varepsilon \sum_{k<m} L_k)$, with tolerance scheduling and a continuation loop.
- **GradNorm** (Wang et al. 2023) — EMA gradient-norm balancing of $w_{\text{pde}}/w_{\text{bc}}/w_{\text{ic}}$, refreshed every 100 steps.
- **RBA** (arXiv:2307.00379) — residual-based attention, pointwise $\lambda \leftarrow 0.999\lambda + 0.01|r|/\max|r|$; an alternative to causal weighting, frozen during L-BFGS.
- **SOAP** (arXiv:2409.11321) — Adam in the Shampoo eigenbasis, 2D parameters only, eigendecomposition refreshed every 10 steps. Implemented for both backends.
- **Two ways to handle the IC** — either a hard-constraint ansatz $\hat{u} = g(x)e^{-(15t)^2/2} + \tanh^2(25t)\,\mathrm{NN}(x,t)$, exact at $t=0$; or a soft IC loss with warm-up and a pre-scale of 15 000 that prevents IC gradient starvation early in training.
- **Checkpoint selection on a stationary metric** — unweighted per-chunk PDE residual + BC + scaled IC, *not* the weighted training loss, which moves as the weights move.

3.3 The sweep

528 runs = roughly 50 architecture configurations × 3 materials × {hard ansatz, soft IC} × {Adam+L-BFGS, L-BFGS-only}, with the SOAP and RBA variants excluded from the L-BFGS-only half (with no Adam phase they would duplicate the base configuration).

`wave/sherlock.sh` is the entire cluster story in one file: it submits itself as a SLURM array job and, on first use, builds the Python environment under a `flock` so that concurrent array tasks wait rather than racing `pip`.

3.4 Status

The infrastructure is complete and the sweep runs, but **its results are not in this repository** — they land on cluster scratch. What is present from the coordinate-network side:

- `final/` — the earlier standalone version of the project: trained checkpoints for vanilla / Fourier / PirateNet / FNO on the three materials, rendered comparisons, heatmaps, residual maps and animations for three experiments, and a `Numerical methods/` folder with the FD solver comparison it was validated against.
- `wave/hyperparam_tuning/` — PINN and PIKAN tuning drivers plus a results-viewing notebook.

4 Arm B — neural operators

Three architectures learn $[E(x), \rho(x), g(x), x, t] \mapsto u(x,t)$, supervised on FD solutions, with no PDE residual in the loss. 512 samples on a 64×64 grid, 410 train / 102 validation, 400 epochs, AdamW with a one-cycle schedule, one H100. The metric is relative L_2 in physical units, which has a useful calibration property: **a model that outputs zeros scores** $\approx 100\%$.

- **FNO** — 2D spectral convolution over the joint (x,t) spectrum.
- **DeepONet** — branch × trunk inner product; a rank-p separable expansion.
- **PFNO** — *paralleled* FNO (not physics-informed): one small 1D FNO per temporal frequency bin, reassembled by inverse rFFT; 33–41 independent branches.

4.1 The unforced arm, and the failure that started it

The original animations showed noise. The cause was not the plotting code — the models were **untrained**, and the plots were faithfully showing that. The configuration was 16 samples, 12 epochs, batch 4: about 39 gradient steps in total, at 10–30 k parameters. FNO scored 81.9%, PFNO 94.8%, DeepONet 97.3% — that is, near-zero fields.

Three changes fixed it, in order of importance: **512 samples instead of 16** (FD generation costs about two seconds, so the small dataset was never a compute constraint), **OneCycleLR** (at a flat learning rate these models sit at normalized MSE ≈ 1.0, exactly what predicting the mean achieves), and greater capacity with a 400-epoch budget.

Model	Parameters	Train time	Val relative L_2	was
FNO	7.39 M	73 s	**2.07%**	81.9%
PFNO	1.23 M	819 s	**4.26%**	94.8%
DeepONet	0.89 M	25 s	**14.43%**	97.3%

Per material family the ordering is identical for all three models, and matches physical intuition — error grows with the number of interfaces, because each interface adds a reflected and a transmitted wave the operator must place correctly:

Family	FNO	DeepONet	PFNO
homogeneous	0.13%	6.07%	0.74%
smooth	0.93%	10.94%	2.59%
two-layer	1.86%	16.22%	3.96%
layered (3–7)	5.28%	23.78%	9.55%

4.2 The target was not converged

The datasets above solve the FD reference on the *same* 64-point grid as the output. Regenerating with identical materials (the `inputs` arrays are bit-identical) and only a `refine=8` target moves the targets by 9.72% on average — and by 7.91% even on homogeneous samples, which contain no interfaces at all. That part is not interface error: it is numerical dispersion of the initial pulse itself, whose width spans only about three cells at 64 points.

Model	vs `refine=1` target	vs `refine=8` target
FNO	2.07%	**1.96**%
PFNO	4.26%	**4.03**%
DeepONet	14.43%	**24.57**%

FNO and PFNO barely move: discretization error is a deterministic function of the material rather than noise, so a model with enough capacity simply learns whichever target it is given and scores the same against it. **DeepONet nearly doubles.** The under-resolved target is smoother — dispersion smears the pulse — and a rank-p separable expansion represents smooth fields far more cheaply than sharp ones. One architecture's headline number was materially flattered by a numerical artefact, and the effect was invisible until the target was fixed.

4.3 The forced arm: a source term and physics residuals

A second, independent arm drives the wave with a Ricker point source instead of an initial pulse, with quiescent initial conditions and t extended to 2. The extension matters: at $t_{\max} = 1$ about 99% of the energy is still inside the domain, so the absorbing boundary is barely exercised and a BC residual would measure nothing; at $t_{\max} = 2$, 99.8% has left. Three inputs now vary — $E(x)$, the source position and the peak frequency — making the problem much harder: FNO 17.13%, PFNO 31.89%, DeepONet 91.82%.

Initial- and boundary-condition residuals were added as **diagnostics** (the loss is still plain supervised MSE), with the FD reference's own residual as the floor, and the metric validated on two controls: a standing wave scores exactly 1.0000, an all-zero field scores 0.

	IC displacement	IC velocity	BC absolute	BC relative
FD reference	0	3.9×10^{-4}	0.886	0.126
FNO	4.0×10^{-3}	0.183	1.500	0.249
PFNO	3.2×10^{-2}	1.185	3.691	0.515
DeepONet	3.3×10^{-2}	0.269	0.123	**0.144**

The result worth keeping is the last row. **DeepONet reports the best boundary score** — an absolute residual seven times *smaller* than the reference's — while being by far the worst model. Decomposing the two terms that must cancel at the boundary shows its boundary terms are about six times smaller than the reference's: there is barely a wave arriving, so it satisfies the radiation condition by having nothing to radiate. **These residuals are only interpretable alongside the field error.**

4.4 Making the physics an actual training loss

Here the residuals enter the objective,

$$L \;=\; \mathrm{MSE}(\hat{u}, u) \;+\; \lambda \left(L_{\mathrm{ic},u} + L_{\mathrm{ic},u_t} + L_{\mathrm{bc}}\right),$$

with the numpy diagnostics reimplemented in torch — the two agree to $\sim 10^{-8}$, verified before any result below was trusted — and each term divided by a fixed precomputed scale so that a single λ balances all three. Setting $\lambda = 0$ reproduces the supervised run exactly.

λ	Field error	IC displacement	IC velocity	BC relative
0 (baseline)	17.13%	4.03×10^{-3}	0.1835	0.2494
0.003	17.02%	3.47×10^{-3}	0.1507	0.2380
0.03	16.92%	2.04×10^{-3}	0.0884	0.1928
0.3	15.89%	1.36×10^{-3}	0.0461	0.1205
1.0	14.84%	5.15×10^{-4}	0.0171	0.0657
3.0	14.20%	7.56×10^{-4}	0.0151	0.0426
10.0	**12.70%**	$\mathbf{1.95\times10^{-4}}$	**0.0058**	**0.0291**

Every metric improves monotonically and **no trade-off appears** — field error falls 26% relative, the initial-velocity residual improves 32-fold. That is unusual enough to suspect, so the collapse check was repeated: at $\lambda = 1$ the field RMS is $0.982\times$ the reference against $0.969\times$ for the supervised baseline, so the model is *more* faithful in amplitude and the gain is real. The likely cause is regularization — with 410 training samples the constraints supply information the data alone does not pin down.

At $\lambda = 1$ the three architectures do three different things:

- **FNO improves on every axis** (17.13 → 14.84%). Its BC residual (0.066) falls below the FD reference's own 0.126, which reflects the reference's first-order Mur discretization rather than the network being "better than truth".
- **PFNO is essentially unchanged** (31.89 → 31.43%). It was predicted to benefit most, since its initial-velocity residual was its clearest weakness. The prediction was wrong, and the reason is the more interesting result: $u_t(x, 0) = 0$ requires 41 independent frequency branches to agree in phase, and a gradient on the assembled field cannot repair a coordination failure spread across 41 sub-networks. The defect is architectural, not an optimization shortfall.
- **DeepONet gets worse** (91.82 → 93.55%). Zero satisfies every one of these homogeneous constraints exactly, so for a model that already cannot fit the data the physics terms are a gradient pointing *toward* the trivial solution.

4.5 Real velocity models

The synthetic sampler — tanh steps and sums of sinusoids — is replaced by 1D depth columns from three published benchmark models (Marmousi, SEG/EAGE Overthrust, SEG/EAGE Salt), keeping the problem, solver, architectures and schedule identical. Two methodological changes are forced by real data:

- **Coarsening by Backus averaging** — arithmetic mean of ρ, harmonic mean of the modulus — the correct long-wavelength effective medium for normal-incidence 1D propagation. Point-sampling a 739-cell Marmousi column onto 64 points would alias the interfaces into noise: 3.2% of its cells jump by more than 200 m/s.
- **A spatial train/validation split.** Neighbouring Marmousi traces are 4 m apart and differ by 0.2% on average, so a random split puts near-duplicates of training profiles into validation and measures interpolation rather than generalization. Validation instead takes four contiguous trace blocks with 320 m buffers discarded on each side.

Arm	Heterogeneity	FNO	PFNO	DeepONet
synthetic	0.068	**1.96%**	**4.03%**	**24.57%**
Overthrust	0.172	**3.59%**	**5.65%**	**26.11%**
Salt	0.097	**6.95%**	**9.30%**	**18.55%**
Marmousi	0.280	**12.33%**	**16.15%**	**39.97%**

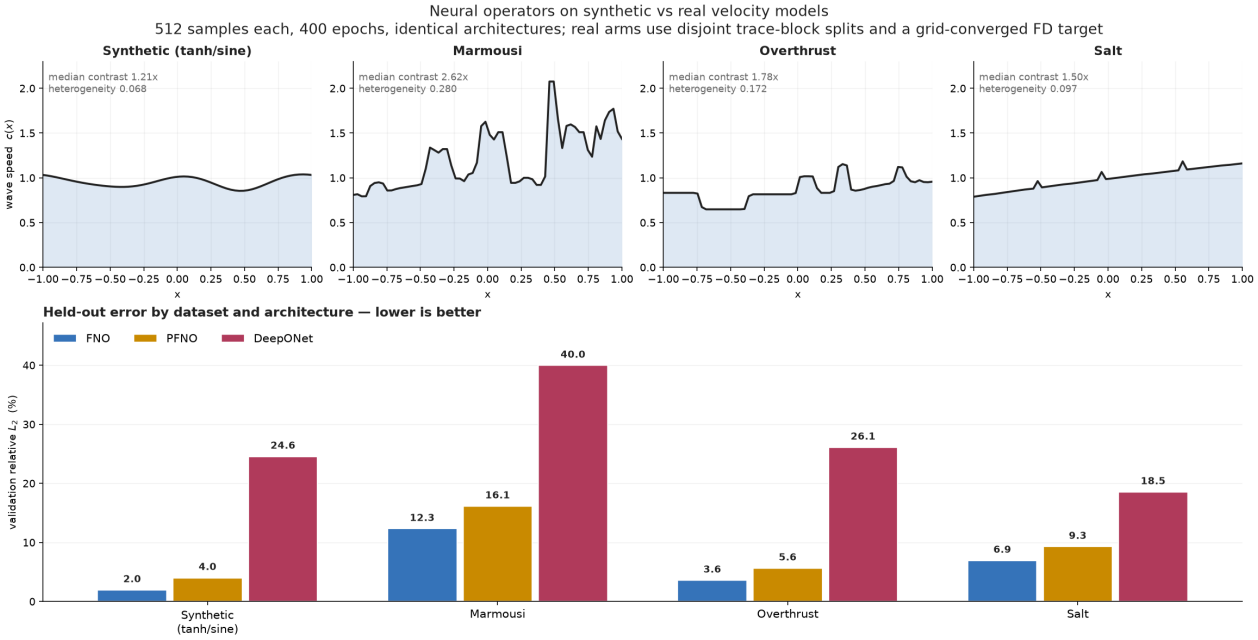

Figure 1: Held-out error by dataset and architecture. The ranking FNO < PFNO < DeepONet holds on every arm.

("Heterogeneity" is the median within-sample standard deviation of $c(x)$.)

The architectural ranking survives on real geology — FNO < PFNO < DeepONet on every arm without exception, which is the main thing this section was run to check. But the margins compress: FNO beats PFNO by 2.1× on synthetic data and only 1.3× on Marmousi. And difficulty tracks **heterogeneity, not contrast**. Marmousi and Overthrust actually get *easier* as $V_{p,\max}/V_{p,\min}$ rises, while Salt reverses completely, because its high-contrast columns are exactly the ones intersecting a salt body — one sharp isolated reflector, which is a different and harder problem than a wide velocity range.

4.6 Visualization

Twenty animations in `simulations/` (4 synthetic, 9 real, 4 forced, 3 superseded), each a 2×4 panel: the velocity model and three line-plot comparisons on top, the FD reference field and three predicted fields below, all on a shared colour scale. Three defects in the original figure were fixed, because they mattered for interpretability independent of model quality:

1. **The reference is now shown.** The original compared the three predictions against each other only — actively misleading when models are undertrained, since a near-zero field looks smooth and plausible.
2. **The colour scale comes from the propagating wave** (98th percentile of $|u|$), not the global maximum. The $t = 0$ pulse is about 1.8× taller than anything after it, so scaling to it spent half the colormap on a single frame.
3. **The palette was re-stepped** for colour-vision separability. The original blue/teal pair sat at $\Delta E = 14.0$, below the legibility floor; the worst adjacent pair is now $\Delta E = 16.0$ under protanopia.

Rendering is deliberately split from training: the training script writes no plots, so the GPU host needs no matplotlib.

5 What the whole thing found

1. **Operator learning has a hard data threshold.** Sixteen samples is not a small dataset for this problem; it is below the point where anything is learnable. A few hundred is the threshold, and generating them costs seconds.

2. **A near-zero prediction is the default failure**, and it looks plausible. Every metric here is calibrated so that zero scores $\approx 100\%$, and every surprising result is checked against an amplitude-ratio control.

3. **The architectural ranking is stable and structural.** FNO < PFNO < DeepONet on every material family, every real geological arm, forced and unforced. It follows from what each architecture can represent: a joint (x, t) spectrum, a per-frequency factorization, and a rank-p separable expansion.

4. **Physics losses help a model that already fits, and hurt one that does not.** FNO improves monotonically to $\lambda = 10$; DeepONet is pushed further toward the trivial solution that satisfies every homogeneous constraint exactly.

5. **Residual diagnostics are not standalone evidence.** The worst model had the best boundary residual.

6. **The reference solution is part of the experiment.** An unconverged target left one architecture's headline number ten points too good.

7. **Difficulty is about sharp isolated reflectors, not velocity range.** $V_{p,\max}/V_{p,\min}$ predicts the wrong sign on two of the three real models.

6 Repository map

Code.

Path	Role
`wave/fd_solver.py`	leapfrog FD reference (conservative form, Mur ABC)
`wave/wave_problem.py`	torch-free problem definition: IC, material sampler, channel layout
`wave/materials.py`, `problem_data.py`	Homogeneous / TwoLayer / MultiLayer + JAX methods; ansatz, FD dispatch
`wave/run_experiment.py`	Arm A runner — argparse, SLURM, backend switch, `MODEL_CONFIGS` / `CONFIG`
`wave/sherlock.sh`	the one cluster script (self-submitting, self-bootstrapping environment)
`wave/run_fno_baseline.py`	original FNO/CNN baseline; now re-exports `wave_problem`
`wave/operator_sim/`	Arm B: dataset generation (synthetic / real / source), the three models, training, physics losses and metrics, rendering
`src/`	Arm A internals — models, optimizers, training loops, losses, PyTorch and JAX
`kaggle/run_fno_t4.py`	T4 preset launcher (smoke / medium / full)

Data and results.

Path	Contents
`operator_data/*.npz`	five datasets, $512 \times 5 \times 64 \times 64$ in / $512 \times 1 \times 64 \times 64$ out (∼8.5 MB each): synthetic `refine=1`, synthetic `refine=8`, Marmousi, Overthrust, Salt, plus the forced arm at 64×80
`operator_data/raw/`	raw velocity models (∼700 MB, gitignored)
`server_outputs/`	unforced synthetic run: predictions, summary, per-epoch histories, log
`server_outputs_real/`	the four real-model arms (`synthetic_r8`, `marmousi`, `overthrust`, `salt`)
`server_outputs_source/`, `server_outputs_pinn/`	forced arm, supervised and physics-informed (with `lambda_sweep.json`)
`simulations/`	20 GIFs — synthetic, `real/`, `source_term/`, `superseded_undertrained/`
`fno_t4_{smoke,medium,full}/`, `operator_results_*`, `kaggle_outputs/`	earlier baseline runs that bracket the data threshold
`final/`	the earlier standalone project: checkpoints, exp1–3 results, numerical-methods comparison

Model checkpoints (`FNO_state.pt` alone is 59 MB) stayed on the training host and were not copied back; predictions and summaries were, which is everything the renderer and the analysis need.

Writing and reference material.

Path	Contents
`docs/operator_simulations.*`	the full Arm B write-up (17 sections), Markdown + LaTeX + PDF, with 11 figures
`notebooks/`	end-to-end FNO/DeepONet/PFNO notebook (`QUICK_RUN` toggle; uses pretrained predictions when present)
`ppt/`	`FNO_Technical_Reference` (LaTeX + PDF), KAN guide, experiments deck, and `diagrams/` — TikZ sources for spectral convolution, FNO2d, the Fourier block, ansatz anatomy, the data pipeline and architecture evolution
`presentations/`, `outputs/`	neural-operator architecture decks and the FNO explainer, several revisions
`videos/`	two rendered FNO explainer videos
`papers/`	∼30 reference PDFs and a BibTeX file, with a curated seven-paper `FNO_reading_order/`
`fno paper/`	the three papers the Arm B implementations follow

7 Status and honest limits

Done and measured. The entire operator arm — four material regimes, forced and unforced, supervised and physics-informed, with converged targets, spatially disjoint splits, calibrated metrics, animations, and a full write-up.

Built but not reported here. The 528-run PINN/KAN sweep. The code, both backends, all seven architectures, SOAP / RBA / causal weighting / GradNorm and the cluster harness are complete; the results live on cluster scratch rather than in the repository, and no cross-architecture comparison has been written up from them.

Limits to keep in view when reading Arm B's numbers.

- **One seed, one split, no error bars.** Tenths of a percent between configurations mean nothing; the order-of-magnitude gaps between architectures do.
- **Fixed initial condition** ($\sigma_g = 0.1$, $x_0 = 0$) — the unforced arm learns a velocity-model → wavefield map, not a general initial-condition → wavefield map.
- **Fixed** 64×64 **grid.** FNO and PFNO are in principle discretization-invariant; zero-shot super-resolution was never tested.
- **In-distribution evaluation only.** No arm is evaluated on a *different* arm — nothing trains on Marmousi and tests on Salt, which is the transfer question a practitioner would actually ask. Transfer to Arm A's canonical materials is also untested.
- **"Real" means published benchmark model, not measurement.** Marmousi and Overthrust are themselves synthetic constructions, geologically realistic but not logged rock; the wave field is FD-simulated in every arm. Only Marmousi ships a density model.
- **Parameters are not matched.** FNO has 7.4 M against DeepONet's 0.89 M and PFNO's 1.2 M, so some of FNO's advantage is capacity; a parameter-matched comparison has not been run. Note also that `train_operators.py` defaults to a smaller DeepONet (`-don-latent 192 -don-hidden 384`) than the 256/512 pair every DeepONet number here uses — re-running without those flags silently produces a smaller, non-comparable model.
- **These are readable reimplementations**, sized to train in minutes, not faithful reproductions of the Li et al. or Lu et al. configurations.

Version-control state. Most of the work described in Section 4 — `wave/operator_sim/`, `docs/`, `notebooks/`, `simulations/`, all `server_outputs*` directories, and the `ppt/` technical reference and diagrams — is currently **untracked**. Only the FNO baseline and the Kaggle runner have been committed.